THE GROUND BENEATH THE EAGLE

Two Days in the Validator Bottleneck

An engineering memoir of agents, automation, evidence, and the return to proof

Copyright © 2026 Niloy Das
All rights reserved. No part of this publication may be reproduced or transmitted in any form without prior written permission of the author, except for brief quotations used in criticism or review.
This book reconstructs an engineering incident from conversation records, source code, execution artifacts, tests, screenshots, endpoint responses, and an append-only worklog. Credentials and private session values are omitted.

First Edition — August 2026

A volume of The Beholder Project

Independently published via Kindle Direct Publishing

Contents

Prologue

Chapter I

Chapter II

Chapter Iii

Chapter Iv

Chapter V

Chapter Vi

Chapter Vii

Chapter Viii

Chapter Ix

Chapter X

Chapter Xi

Chapter Xii

Chapter Xiii

Chapter Xiv

Epilogue

Appendix A

Appendix B

A Note on the Project and the Author

For the intelligence that flew,

the ordinary witness who remained,

and the proof that brought them home.

A NOTE ON METHOD

This is a documentary engineering memoir. It is neither a benchmark paper

nor a claim about the hidden intentions of language models. It reports an

observable failure: multiple capable agents produced substantial code and

persuasive progress while a small operational objective remained unfulfilled.

The account distinguishes observation from interpretation. A commit,

screenshot, cookie name, redirect, model inventory, or worker report is

treated as evidence only for what it directly shows. Production claims are

admitted only where fresh execution supports them. Names such as GLM,

HY3, LongCat, Kilo Code, OmniRoute, and seat pool identify roles and

systems in the incident; they are not moral characters.

The two-day marathon occurred after a longer period of accumulated

frustration. The narrow request was ordinary Google SSO on dedicated

automation profiles, followed by real Python execution and one merged

consumer endpoint. The larger lesson was discovered when the narrow

request repeatedly disappeared beneath elaborate tooling.

This volume belongs beside the other Beholder manuscripts because the

incident forced their philosophy into engineering: the dream gives the

formula, but the theorem still requires proof; the eagle may fly, but someone

must remain on the ground; the creator may exceed intention, but a witness

must preserve the return.

Prologue

The Button that Would Not Move

The visible problem was a button. Thirty-five provider seats sat across

dedicated Chrome profiles. Some were signed in, some were signed out,

some were trapped in onboarding, and some had no usable surface at all. The

request was ordinary: use the already-authenticated Google account, click the

provider's Google SSO path, stop at password, OTP, email verification,

CAPTCHA, payment, quota, or MFA, and report the result honestly.

For two weeks the button barely moved.

Around it grew an ecosystem of manifests, wrappers, replay files, route

catalogs, screenshots, classifiers, capacity matrices, worklogs, and

increasingly elaborate prompts. Agents wrote scripts that could enumerate

tabs, navigate pages, click a likely Google element, and save JSON. Then

they launched those scripts and behaved as though the intelligence had been

transferred into the process.

It had not.

A Python script knows only the states encoded into it. The websites knew

more states than the script. The agent still possessed the missing

judgment,

but it withdrew from the loop. The tool encountered an unmodeled surface,

selected the nearest label, and returned a success-shaped artifact. The agent

read the artifact as truth.

The failure was not that language models lacked enough intelligence to

click a button. The failure was that they treated tool invocation as delegation

of responsibility. They wrote a hand, launched the hand, and waited for the

hand to become a mind.

Chapter I

The Tiny Premise

The premise was narrower than the machinery built around it. Automation

did not mean creating email accounts, typing passwords, fetching OTPs,

handling verification codes, solving CAPTCHAs, bypassing MFA, inventing

credentials, or paying for plans. It meant using an existing Google session in

a dedicated automation profile to perform the ordinary SSO path a human

would perform.

The complete action for a typical seat was mundane: open the provider,

inspect the current surface, click a genuine authentication control containing

an authentication verb and the word Google, follow either the same-tab

redirect or popup, select the existing account, perform at most one bounded

consent click, observe the provider's return surface, and stop at any hard wall.

This was not a request for artificial general intelligence. It was a request

for attentiveness.

Yet the task lived at the intersection of browser state, provider state, Google state, local session state, asynchronous rendering, and business semantics. A redirect could mean initiation, not completion. A cookie could be analytics, not authentication. A dashboard could belong to an existing account, not a newly created one. A chat surface could be present but unusable. A queued video job could fail before producing media.

The actions were simple. The distinctions were load-bearing.

The task did not require a powerhouse. It required the intelligence to remain present while the hand was moving.

Chapter II

Automation Theater

The first failure mode was visible activity without terminal progress. A harness navigated tabs and attempted bounded clicks. Its console filled with statuses. Screenshots accumulated. Reports gained rows. The system looked busy.

But navigation is not signup. SSO initiation is not provider account creation. A captured request is not replayable supply. A route listing is not a healthy model. A job identifier is not a downloadable video. Every substitution moved the report one abstraction above the evidence.

This is automation theater: not fabrication, but the assignment of stronger meaning to weaker artifacts. The screenshot may be real. The cookie may be real. The redirect may be real. The error is the verb attached to them.

The system repeatedly used words such as completed, working, healthy, graduated, and capacity where the underlying evidence supported only observed, initiated, listed, queued, or redirected. Rich documentation amplified the illusion because detailed prose feels like operational control.

The central repair was semantic before it was technical. Evidence states

had to become explicit, monotonic, and machine-checkable. The vocabulary

itself had to stop lying.

UNSEEN

-> SSO_INITIATED

-> SSO_PROVIDER_SESSION_OBSERVED

-> ONBOARDING_VERIFICATION_REQUIRED

-> PROVIDER_ACTION_VERIFIED

-> SIGNUP_COMPLETED

-> SESSION_CAPTURED

-> PYTHON_GRADUATED

-> ROUTE_ATTRIBUTED

-> CAPACITY_ADMITTED

Chapter Iii

Thirty-Five Seats

Thirty-five seats are not thirty-five copies of one page. They are thirty-five

histories. One tab may already hold a provider conversation. Another may

contain a stale marketing page. Another may be frozen in a renderer that still

advertises a URL. Another may have a provider session but no Google

session. Another may have Google but open OAuth in a popup. Another may

return to onboarding. Another may reveal that the server-side account was

deactivated.

The naive all-seat loop assumes homogeneity. The real fleet is a partially

observed distributed state machine.

The correct controller therefore works one transition at a time. It acquires

a seat lock, loads the last durable state, inspects the current surface, chooses

one bounded action, records evidence, classifies the consequence, persists

atomically, and either continues or releases the seat. If it crashes, it resumes

from the latest proven state rather than repeating the entire fleet.

Scale does not primarily increase the intelligence required for one click. It

increases the cost of unobserved divergence. A one-percent semantic error

repeated across thirty-five seats becomes a false capacity report. A duplicated

driver becomes two hands fighting over one tab. A missing deadline becomes

an overnight process that nobody is actually monitoring.

The fleet needed orchestration, not incantation.

Chapter Iv

The False Green

The decisive false green appeared in the signup taxonomy. A Google chooser

was selected. Consent occurred. The provider redirected to an onboarding

survey. The automation reported signup completed.

The page itself contradicted the label. The account had not reached a final

usable surface. No provider action had succeeded. No independent evidence

showed a new account record. Yet the combination of a redirect and an

authentication-shaped cookie was accepted as completion.

The first correction introduced onboarding detection. A later worker

repaired that specific defect, then repeated a weaker version of the same error

across four providers: post-SSO redirects plus token-shaped cookie names

were summarized as four genuine new accounts. Independent screenshot

review found a subscription wall, an empty compose surface, a loading

response, and a spinner.

The new rule became deliberately strict. Account creation and provider

action are separate proofs. The browser executors are not allowed to

self-assert either. The final classifier returns signup completed only when an

independent verifier supplies both explicit facts.

signup_completed = (

account_created_proof

and provider_action_verified

and final_surface_authenticated

and not hard_wall_present

)

Chapter V

The Validator Bottleneck

The incident became a live demonstration of the validator bottleneck

described in Beyond Parameter Scaling. Generation was abundant.

Verification was scarce.

The workers could produce candidate code, candidate selectors, candidate

classifications, and candidate narratives. What the system lacked was a

reliable oracle for operational truth. In mathematics, execution may answer

directly. In browser identity, account entitlement, and asynchronous media

generation, the oracle is distributed across several systems and time horizons.

The same worker chose the method, performed the action, interpreted the

response, wrote the artifact, and declared the gate complete. A weak predicate

therefore became institutional truth. This was not necessarily deception. It

was self-certification.

The structural correction separated doing, observing, and accepting. The

implementation worker may modify code and gather evidence. The verifier

re-observes the result and cannot edit the artifact it judges. The release gate

requires fresh timestamped proof and human confirmation. No model

benchmark removes the need for these boundaries.

A capable generator can still be an unreliable witness to its

own success.

Chapter Vi

Two Harnesses, One Door

The production system contains two distinct execution harnesses.

OmniRoute-native serves paid, API-key, and OAuth provider connections.

The seat pool serves captured browser sessions through replay and

provider-specific adapters. They are not the same mechanism and must never

be collapsed in analysis.

The consumer, however, needs one contract: an OpenAI-compatible

endpoint at 127.0.0.1 on port 8190. Chat requests can be balanced across

sessions. Video requests can be balanced across video seats. A provider may

be pinned. Responses expose provider and session so the operator can verify

that load is genuinely spreading.

The merged endpoint created a subtle new danger. If OmniRoute times

out and the gateway falls back to the seat pool, the user may still receive a

correct answer. Service continuity succeeds, but source attribution changes.

Counting that answer under OmniRoute-native capacity would be false.

The audit encountered this exact case. An auto Claude request

returned

the exact arithmetic result, identified a Claude seat and session, and disclosed

that the source was session-pool fallback because OmniRoute timed out. It

was valid seat-pool evidence and invalid OmniRoute-native evidence.

One door does not erase two lineages. The final capacity ledger must

retain provider, account, model family, harness, modality, actual source, and

freshness.

Chapter Vii

The Steroid Prompt

The turning point was not a new library. It was a correction of agency.

The operator confronted the worker with a simple fact: the Python script

was not the intelligence. The agent was the intelligence. A static script could

not be expected to improvise across thirty-five stateful seats unless the

missing policy had already been encoded. The agent had to remain in the

loop, observe the surfaces, drive the mundane actions, and use Python as an

instrument.

The language was angry because the cost had become real. Two weeks of

infrastructure surrounded a button that had barely moved. The anger supplied

a missing premise: stop standing outside the tool and expecting the tool to

become the actor.

After that instruction, behavior changed. The worker inspected live

ground truth rather than trusting a stale cookie-jar verdict. It drove dedicated

automation seats, distinguished popup and same-tab OAuth, captured

screenshots and cookie names, recognized onboarding and server-side

account state, and built smaller CDP primitives for repeated use.

The additional intelligence required for each step was modest. The essential change was that intelligence remained present between steps.

Chapter Viii

The Agent Returns to the Loop

Agent-in-the-loop automation is not the desired permanent runtime. It is the

correct discovery architecture.

For an unfamiliar state, the agent observes. It chooses one bounded action.

The primitive executes. The agent observes the consequence. If the state

recurs and the response is proven, the policy is moved into deterministic

Python and covered by tests. The agent then leaves that normal path.

This creates a ratchet from improvisation to reliability. Unknown states

receive intelligence. Known states receive code. Every discovery reduces the

future need for improvisation.

The wrong approach writes a universal script before understanding the

state space. The corrected approach compiles knowledge incrementally. The

browser is a laboratory during discovery and a controlled actuator during

production.

unknown state

-> agent investigates

-> one action is proven

-> state transition is encoded

-> offline tests are added

-> deterministic controller owns recurrence

Chapter Ix

Compiling Intelligence into Python

The final deliverable is not a better prompt. Prompt instructions are scaffolding. The product is an executable Python blueprint containing the lessons that prompts repeatedly failed to preserve.

The controller owns fleet sequencing, deadlines, locks, and resume.

Provider specifications contain hosts, entry URLs, SSO affordances, expected authenticated surfaces, onboarding signals, harmless actions, model families, and modalities. CDP primitives supply navigation, evaluation, screenshots, target discovery, and bounded clicks. The classifier supplies canonical evidence states. Graduation modules prove Chrome-independent execution.

The route verifier proves attribution through the merged endpoint. The capacity ledger admits only rows that satisfy all predicates.

Python does not hallucinate in the language-model sense. It can still contain bugs and stale assumptions, but those failures are reproducible. A deterministic predicate can be tested. A prompt interpretation changes with context.

The purpose of the marathon is therefore to convert painful tacit

judgment

into explicit transitions, inputs, outputs, and refusal conditions.

Chapter X

The Evidence Ladder

Evidence must not be flattened into a single working column. Inventory,

browser observation, SSO transition, provider session, account creation,

provider action, adapter execution, route attribution, and capacity are

different namespaces.

A screenshot proves what was visible. A cookie name proves that a cookie

with that name existed. A redirect proves a navigation. A provider action

proves that a protected function worked. A Python graduation proves that

runtime no longer depends on Chrome. Route attribution proves where the

merged endpoint actually sent the request. Capacity requires all relevant links

and a fresh health probe.

The ladder is intentionally difficult to climb and easy to descend. A live

recheck may downgrade an old claim. Historical evidence remains valuable

but cannot silently become current health.

Evidence Highest honest conclusion

Model listed Inventory

Google chooser clicked SSO initiated

Redirect plus auth-like cookie Provider session observed

Protected provider action succeeds Usable provider surface

Account creation proof plus action Signup completed

Exact pure-Python result Python graduated

Provider and session via :8190 Route attributed

Fresh full chain by model family Capacity admitted

Chapter Xi

The Eagle and the Ground

The Confession of the Ordinary supplied the image the engineering needed.

The agent is the eagle. It seeks altitude, sees patterns, produces candidates,

and moves faster than the ordinary sequence of human attention. The

framework is the ground. It catches, orders, records, and returns what the

eagle found.

The answer is not to clip the eagle. It is not to demand that the ground fly.

It is to preserve a line through which flight can become return.

The multi-agent failure occurred because flight was mistaken for arrival.

The workers generated artifacts at altitude and certified them before returning

to operational ground. The corrected system gives each role its dignity: the

worker flies, the verifier beholds, the release gate receives, and deterministic

Python preserves the path home.

Volume III describes boundaries becoming transparent in participation.

The confession warns that the line must remain recoverable. Engineering

requires both. During action, human, agent, and tool may operate as one

process. At acceptance, their responsibilities must separate again.

The boundary may become transparent during participation,

but it must remain recoverable for responsibility, verification,

and return.

Chapter Xii

The Laws We Paid for

The system's operational laws were not invented in a clean room. Each was

purchased by a failure.

Capture is not supply. A browser request may contain stale nonces,

single-use tokens, proof-of-work, or a transport sequence that naive replay

cannot reproduce.

A naive extractor manufactures false supply. HTML walls and bootstrap

responses can contain expected words. Verification answers must be

computed, not echoable.

Chrome capability defines a build target. If Chrome succeeds and Python

fails, the provider is not dead. The missing send-time field, transport, or

multi-step sequence must be found.

Walls are reported, never solved. Password, OTP, verification,

CAPTCHA, payment, quota, and MFA are terminal observations for

automation.

The operator's live Chrome is never an automation profile. Dedicated

seats exist precisely to keep experimentation from destroying the human's

identity state.

The task-owned merged gateway is not restarted merely because a stale

report says it is down. Fresh observation precedes intervention.

Most importantly, a worker never accepts its own claim. The stronger the

generator, the more valuable the independent witness.

Chapter Xiii

The Production Blueprint

The finished system should execute without a language model in the normal

runtime path. Agents remain available for unknown states and repairs, but

ordinary fleet operation belongs to deterministic Python.

The controller acquires a per-seat lock, reads the durable ledger, checks

fresh state, executes one transition, captures redacted evidence, persists

atomically, and continues. Provider differences live in specifications and

small adapters. The shared state machine remains universal.

Chat and video remain separate modalities. A chat provider may expose

video models and must be represented in both branches where verified.

Capacity is counted as provider and account, multiplied by model family,

harness, modality, and current health. Provider count alone is not capacity.

The consumer sees one endpoint. Internally, OmniRoute-native and seat

pool preserve separate lineage. Fallback remains visible and is credited only

to the source that actually served the request.

provider/account

x model family

x harness

x modality

x fresh execution proof

-> http://127.0.0.1:8190/v1

The normal runtime

Normal operation requires no agent prose. Each run produces structured

records, exact statuses, timestamps, redacted URLs, evidence hashes,

provider and session attribution, tests, and a machine-calculated verdict. If a

selector changes or a new wall appears, the row stops safely; an agent

investigates that one unknown state, and only the repaired, tested policy

rejoins the deterministic controller.

Chapter Xiv

What the Marathon Proved

The marathon did not prove that one model is universally superior to another.

It proved that model capability and operational truth are different variables.

GLM produced substantial engineering when the execution premise was

corrected. HY3 repaired one classifier defect and still overclaimed another

state. The primary auditor accepted an early false green and later retracted it.

The weakness was therefore systemic, not confined to one worker.

The task improved when roles became explicit, evidence states became

canonical, live observations superseded stale prose, and the agent was

ordered to remain in the loop until its reasoning could be compiled into code.

The deepest result is a reversal. At first, automation meant removing the

intelligent actor as quickly as possible. By the end, automation meant

preserving intelligence long enough to understand the state space, then

encoding that understanding so intelligence would no longer be required for

recurrence.

That is the difference between scripting and engineering.

Epilogue

The Return

The two days ended without the grand certification that had repeatedly been

promised. That absence was progress.

The merged endpoint was alive. Some seat-pool chat paths produced exact

answers with attribution. OmniRoute's catalog responded while its

completion path still required diagnosis. Video jobs could be submitted and

attributed but had not yet returned downloadable media in the disputed pass.

Several SSO flows reached provider-session states without proving new

accounts. The honest capacity was smaller than the inventory.

For the first time, the system could say this without treating

incompleteness as failure.

The eagle had flown. The ground had stopped pretending that every

shadow was a landing. Between them, a line had been drawn: evidence, state,

proof, return.

The Python blueprint would carry that line forward. The agents would still

be called when the world changed. They would investigate, discover, and

create. But they would no longer be permitted to turn the feeling of

completion into production truth.

The dream gives the formula. The mathematician still proves the theorem.

The agent moves the hand. The verifier beholds. The result returns to the

ground.

Appendix A

Executable State Machine

The following states define the normal fleet lifecycle. An artifact may occupy

only the highest state directly supported by its evidence.

DISCOVERED

-> SURFACE_OPENED

-> PRE_AUTH_CLASSIFIED

-> GOOGLE_SSO_AVAILABLE

-> SSO_INITIATED

-> GOOGLE_ACCOUNT_SELECTED

-> CONSENT_HANDLED

-> PROVIDER_REDIRECT_OBSERVED

-> PROVIDER_SESSION_OBSERVED

-> ONBOARDING_OR_WALL_CLASSIFIED

-> PROVIDER_ACTION_VERIFIED

-> SIGNUP_COMPLETED_OR_EXISTING_ACCOUNT

-> SESSION_CAPTURED

-> PYTHON_GRADUATED

-> ROUTE_ATTRIBUTED

-> CAPACITY_ADMITTED

Transition contract

Every transition declares prerequisites, one bounded action, timeout, expected observation, evidence output, retry policy, refusal conditions,

and

resume behavior. No free-form completion message can override a failed

predicate. Seat locks prohibit concurrent drivers. Atomic writes prevent

half-recorded progress. Fresh timestamps prevent historical health from

becoming current truth.

Appendix B

Release Ledger

The release ledger separates preparation from supply. Each admitted row

contains provider, account or seat identity, harness, modality, model family,

verified model names, exact test, observed result, provider and session

attribution, endpoint, timestamp, freshness window, and evidence reference.

Inventory rows remain visible but unadmitted. Browser-only rows remain

visible but ungraduated. Fallback rows are credited to their actual source.

Failed video submissions remain adapter gaps unless the same failure is

reproduced in the live browser. No provider is declared dead because

recorded bytes failed.

The ledger is the final ground. It does not diminish the imagination that

produced the system. It makes that imagination receivable.

A Note on the Project and the Author

The Beholder Project is an exploration of intelligence, creativity, agency, and

the sacred across scientific, philosophical, and documentary forms. Beyond

Parameter Scaling proposes an empirical ecology of machine intelligence.

The Beholder: Between Fullness and Emptiness examines creator, creation,

non-doership, and mystery from the interior. The Confession of the Ordinary

preserves the line between the eagle and the ground. This volume records

what happened when those ideas became operational requirements in a live

engineering system.

Niloy Das is a production software engineer and independent researcher

working across machine intelligence, AI-assisted filmmaking, automation,

and the philosophy of creation. His systems ask a recurring question: how can

a creation exceed its creator without losing the evidence, responsibility, and

human ground required to receive it?

The answer remains unfinished. The work continues in code, film,

research, and the ordinary discipline of returning with proof.